



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Отчет по рубежному контролю №1
«Технологии разведочного анализа и обработки данных»
по дисциплине «Технологии машинного обучения»
Вариант №28**

Выполнил:
студент группы ИУ5Ц-84Б
Перовошиков Н.Д.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2024 г.

СОДЕРЖАНИЕ ОТЧЕТА

1. Примечание	3
2. Дополнительные требования по группам	3
3. Листинг	3
3.1. Подключение библиотеки и импорт данных.....	3
3.2. Изучение данных	4
3.3. Анализ выбросов (Ящик с усами).....	6
3.4. Пропуски	7
3.5. Дубликаты	7
3.6. Обработка категориальных признаков	7
3.7. Создание нового признака.....	8
3.7.1. Группировка по возрастам	8
3.7.2. Группировка по уровню частоты сердечных заболеваний.....	9
3.8. Частота сердечных сокращений в разрезе пола	10
3.9. Распределение пола и наличия сердечных заболеваний	11

1. Примечание

Если в Вашем наборе данных отсутствуют данные, необходимые для решения задачи, создайте их искусственно. Например, если отсутствуют категориальные признаки, создайте категориальный признак на основе числового. Если отсутствуют пропуски, замените на пропуски часть значений в одном или нескольких признаках.

Также Вы можете дополнительно использовать датасеты, содержащие необходимые данные, например использовать дополнительный датасет, содержащий пропуски.

2. Дополнительные требования по группам

Для студентов группы ИУ5-64Б, ИУ5Ц-84Б - для произвольной колонки данных построить график "Скрипичная диаграмма (violin plot)".

Задача №4

Для заданного набора данных постройте основные графики, входящие в этап разведочного анализа данных. В случае наличия пропусков в данных удалите строки или колонки, содержащие пропуски. Какие графики Вы построили и почему? Какие выводы о наборе данных Вы можете сделать на основании построенных графиков?

Наборы данных:

<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

3. Листинг

3.1. Подключение библиотеки и импорт данных

```
# Подключение библиотеки
import pandas as pd
import numpy as np
import seaborn as sb
import matplotlib.pyplot as plt
```

```
# Загрузка датасета
try:
    df = pd.read_csv('heart.csv', delimiter=',')
    print('Загружен датасет')
except Exception as ex:
    print('Отсутствует датасет. Проверьте путь файла')
    print('Error:', ex)
```

Загружен датасет

3.2.Изучение данных

```
# Выводим информацию
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         1025 non-null   int64
 1   sex         1025 non-null   int64
 2   cp          1025 non-null   int64
 3   trestbps    1025 non-null   int64
 4   chol        1025 non-null   int64
 5   fbs         1025 non-null   int64
 6   restecg     1025 non-null   int64
 7   thalach     1025 non-null   int64
 8   exang       1025 non-null   int64
 9   oldpeak     1025 non-null   float64
10   slope       1025 non-null   int64
11   ca          1025 non-null   int64
12   thal        1025 non-null   int64
13   target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```
# Первые 5 строк
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
# Последние 5 строк
df.tail()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

```
df.describe()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268	0.529756	149.114146	0.336585	1.071512	1.385366	0.754146	2.591268	0.591268
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175053	0.617755	1.030798	1.591268	0.488372
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000	1.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000	2.000000	0.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000	3.000000	0.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.000000	1.000000

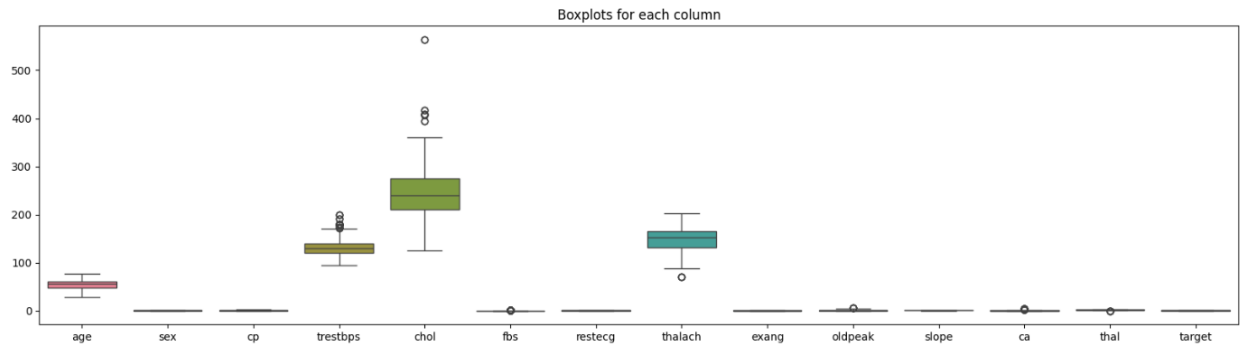
Анализ описательной статистики

- **age:** Средний возраст составляет около 54 лет. Минимальный возраст в данных - 29 лет, а максимальный - 77 лет.
- **sex:** Примерно 70% данных относятся к мужчинам.
- **cp:** Значения от 0 до 3, представляющие различные типы боли в груди.

- **trestbps:** Среднее кровяное давление около 131.6 мм рт.ст. Минимальное значение - 94 мм рт.ст., а максимальное - 200 мм рт.ст.
- **chol:** Средний уровень холестерина составляет 246 мг/дл. Минимальное значение - 126 мг/дл, а максимальное - 564 мг/дл.
- **fbs:** Примерно 15% людей имеют уровень сахара в крови натощак выше 120 мг/дл.
- **restecg:** Значения от 0 до 2, представляющие результаты электрокардиографии в состоянии покоя.
- **thalach:** Средняя максимальная частота сердечных сокращений около 149 ударов в минуту. Минимальное значение - 71 удар в минуту, а максимальное - 202 удара в минуту.
- **exang:** Значения 1 и 0, указывающие на наличие или отсутствие стенокардии, вызванной физической нагрузкой.
- **oldpeak:** Среднее значение депрессии ST около 0.34. Минимальное значение - 0, а максимальное - 6.2.
- **slope:** Значения от 0 до 2, представляющие наклон пика упражнений ST при нагрузке.
- **ca:** Значения от 0 до 3, указывающие на количество крупных сосудов.
- **thal:** Значения от 1 до 3, представляющие различные состояния сердечной недостаточности.
- **target:** Значения 1 и 0, указывающие на наличие или отсутствие сердечного заболевания.

3.3. Анализ выбросов (Ящик с усами)

```
plt.figure(figsize=(20, 5))
sb.boxplot(data=df)
plt.title("Boxplots for each column")
plt.show()
```



Выбросы имеются у колонок trestbps, chol, fbs, thalach, oldpeak, ca, thal.

```
# Скорпируем
df_data = df.copy()
```

```
# Очистим
df_data[df_data['trestbps'] > (df_data.describe()['trestbps']['25%'] - df_data.describe()['trestbps']['std'])]
df_data[df_data['trestbps'] < (df_data.describe()['trestbps']['75%'] + df_data.describe()['trestbps']['std'])]

df_data[df_data['chol'] > (df_data.describe()['chol']['25%'] - df_data.describe()['chol']['std'])]
df_data[df_data['chol'] < (df_data.describe()['chol']['75%'] + df_data.describe()['chol']['std'])]

df_data[df_data['fbs'] > (df_data.describe()['fbs']['25%'] - df_data.describe()['fbs']['std'])]
df_data[df_data['fbs'] < (df_data.describe()['fbs']['75%'] + df_data.describe()['fbs']['std'])]

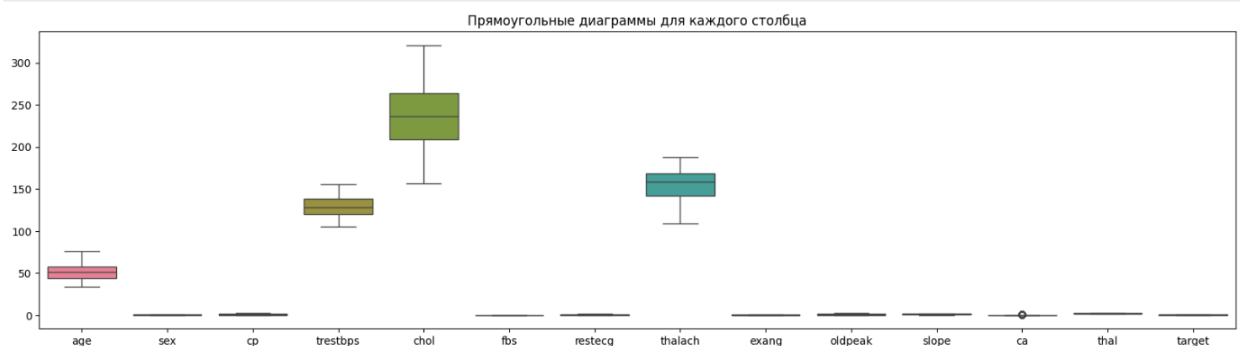
df_data[df_data['thalach'] > (df_data.describe()['thalach']['25%'] - df_data.describe()['thalach']['std'])]
df_data[df_data['thalach'] < (df_data.describe()['thalach']['75%'] + df_data.describe()['thalach']['std'])]

df_data[df_data['oldpeak'] > (df_data.describe()['oldpeak']['25%'] - df_data.describe()['oldpeak']['std'])]
df_data[df_data['oldpeak'] < (df_data.describe()['oldpeak']['75%'] + df_data.describe()['oldpeak']['std'])]

df_data[df_data['ca'] > (df_data.describe()['ca']['25%'] - df_data.describe()['ca']['std'])]
df_data[df_data['ca'] < (df_data.describe()['ca']['75%'] + df_data.describe()['ca']['std'])]

df_data[df_data['thal'] > (df_data.describe()['thal']['25%'] - df_data.describe()['thal']['std'])]
df_data[df_data['thal'] < (df_data.describe()['thal']['75%'] + df_data.describe()['thal']['std'])]
```

```
plt.figure(figsize=(20, 5))
sb.boxplot(data=df_data)
plt.title("Прямоугольные диаграммы для каждого столбца")
plt.show()
```



Результат получился нормальной диаграммой с минимальным выбросом.

3.4. Пропуски

```
df_data.isna().sum()
```

```
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

Пропусков нет, поэтому создаем искусственно по условию задания.

Например, заменим 15% значений в одном или нескольких признаках на пропуски. Предположим, что мы хотим заменить 15% значений в столбце 'trestbps' на пропуски.

```
# Например, заменим 15% значений в одном или нескольких признаках на пропуски.
# Предположим, что мы хотим заменить 15% значений в столбце 'trestbps' на пропуски.
percentage_of_missing_values = 0.15
num_of_values_to_replace = int(len(df_data) * percentage_of_missing_values)
indices_to_replace = np.random.choice(df_data.index, num_of_values_to_replace, replace=False)
df_data.loc[indices_to_replace, 'trestbps'] = np.nan
```

```
# Снова выводим информацию о пропусках
df_data.isna().sum()
```

```
age      0
sex      0
cp       0
trestbps 74
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

3.5. Дубликаты

```
# Подсчитаем количество дублирующих строк
duplicate_count = df_data.duplicated().sum()
print("Количество дублирующихся строк:", duplicate_count)
```

Количество дублирующихся строк: 283

```
# Удаление дублирующихся строк
df_data.drop_duplicates(inplace=True)
```

```
# Повторно проверяем количество дублирующихся строк после удаления
duplicate_count = df_data.duplicated().sum()
print("Количество дублирующихся строк после удаления:", duplicate_count)
```

Количество дублирующихся строк после удаления: 0

3.6. Обработка категориальных признаков

```
df_data['fbs'] = df_data['fbs'].replace({'Правда': 1, 'Ложь': 0})
```

```
df_data['sex'] = df_data['sex'].replace({'Мужчина': 1, 'Женщина': 0})
```

```
df_data['target'] = df_data['target'].replace({'имеют серд.заб.': 1, 'не имеют серд.заб.': 0})
```

```
df_data['exang'] = df_data['exang'].replace({'да': 1, 'нет': 0})
```

3.7. Создание нового признака

3.7.1. Группировка по возрастам

```
df_data['age'].value_counts().sort_index()
```

```
age
34    1
35    5
37    1
39    4
40    5
41   13
42    6
43    8
44   14
45    9
46    7
47    8
48    9
49    4
50   10
51   12
52    4
```

```
def create_age_group(df):
    # Задаем группы возрастов и метки
    age_bins = [34, 44, 55, 66, 76]
    age_labels = ['34-44', '45-55', '56-66', '67-76']

    # Создаем новую колонку 'Age_Group' на основе групп возрастов
    df['age_group'] = pd.cut(df['age'], bins=age_bins, labels=age_labels, right=False)

    return df
```

```
# Создадим
df_data = create_age_group(df_data)
```

```
df_data = df_data.dropna(subset=['age_group'])
```

```
df_data['age_group'].value_counts().sort_index()
```

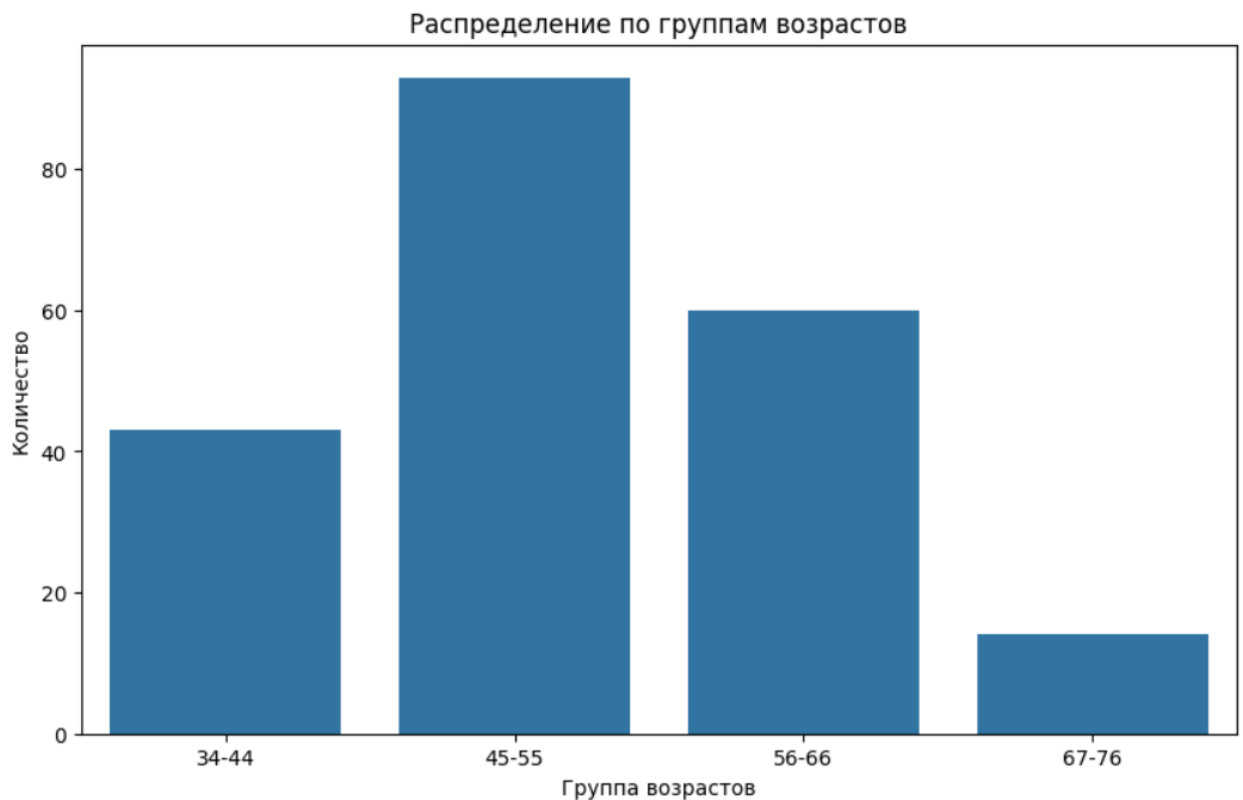
```
age_group
34-44    43
45-55    93
56-66    60
67-76    14
Name: count, dtype: int64
```

```
# Создаем график
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму для групп возрастов
sb.countplot(x='age_group', data=df_data)

# Добавляем заголовок и подписи
plt.title('Распределение по группам возрастов')
plt.xlabel('Группа возрастов')
plt.ylabel('Количество')

# Отображаем график
plt.show()
```



По графику можно сказать, что наибольшее количество людей имеют возраст в диапазоне от 45 до 55 лет, в то время как наименьшее количество людей приходится на возраст от 67 до 76 лет.

3.7.2. Группировка по уровню частоты сердечных заболеваний

```
def categorize_thalach(df, low_threshold, high_threshold):
    df['thalach_level'] = pd.cut(df['thalach'], bins=[float('-inf'), low_threshold, high_threshold, float('inf')],
                                labels=['Низкий', 'Средний', 'Высокий'], include_lowest=True, right=False)
    return df

high_threshold = round(df_data['thalach'].max() - df_data['thalach'].std(), 3)
low_threshold = round(df_data['thalach'].min() + df_data['thalach'].std(), 3)

# Создаем столбец 'thalach_level'
df_data = categorize_thalach(df_data, low_threshold, high_threshold)

# Создаем график
plt.figure(figsize=(10, 6))
sb.countplot(x='thalach_level', data=df_data)

plt.title('Распределение по частотам сердечных сокращений')
plt.xlabel('Уровень частоты сердечных сокращений')
plt.ylabel('Количество')

plt.show()
```



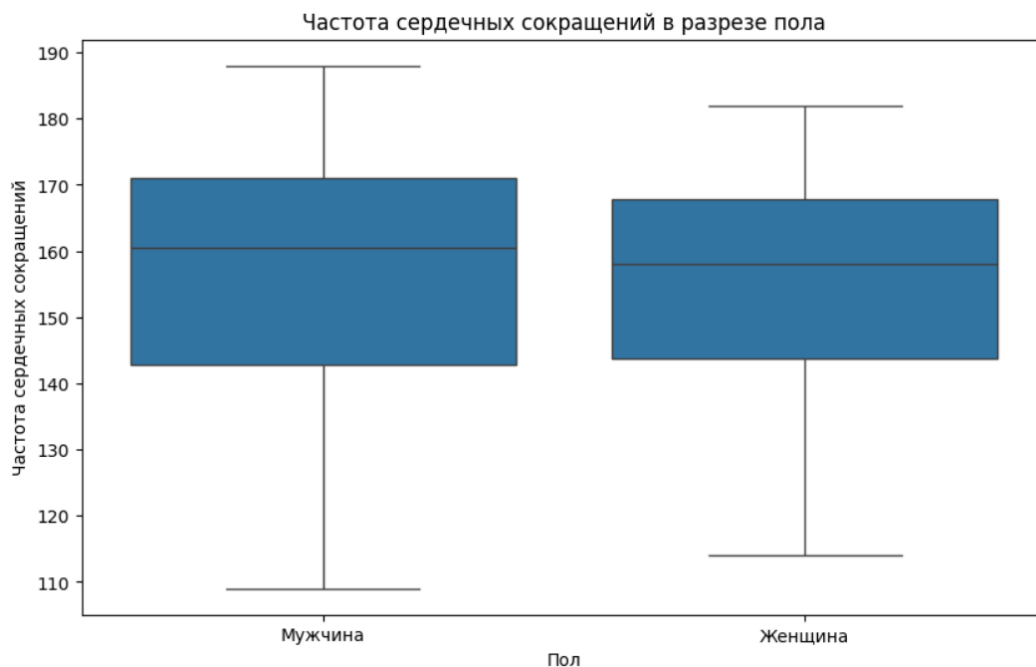
Большинство людей преобладают средний уровень частоты сердечных сокращений.

3.8. Частота сердечных сокращений в разрезе пола

```
# Создаем график
plt.figure(figsize=(10, 6))
sb.boxplot(x=df_data['sex'].replace({1: 'Мужчина', 0: 'Женщина'}), y='thalach', data=df)

plt.title('Частота сердечных сокращений в разрезе пола')
plt.xlabel('Пол')
plt.ylabel('Частота сердечных сокращений')

plt.show()
```

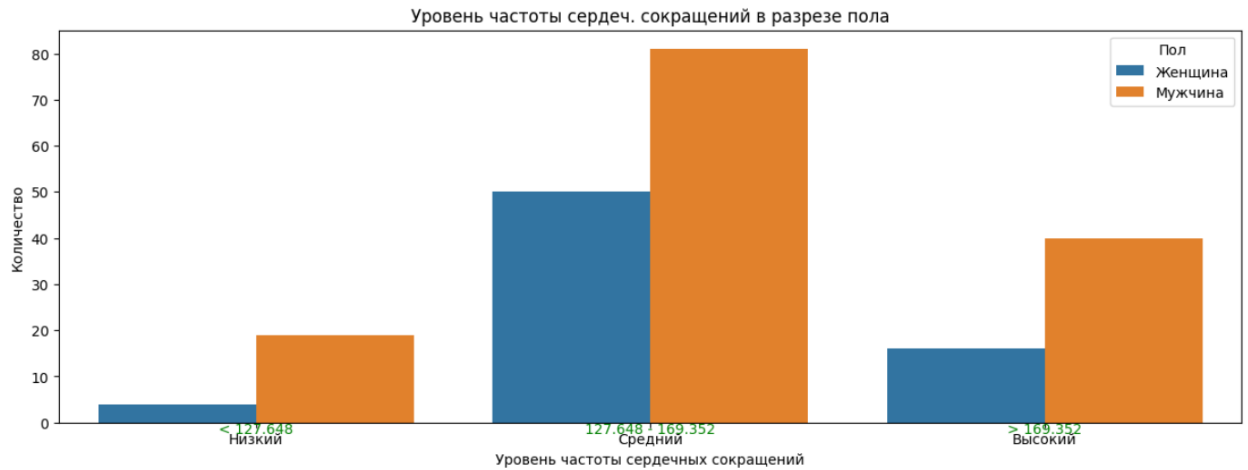


```
# Создаем график
plt.figure(figsize=(15, 5))
sb.countplot(x='thalach_level', hue='sex', data=df_data, order=['Низкий', 'Средний', 'Высокий'])

plt.title('Уровень частоты сердеч. сокращений в разрезе пола')
plt.xlabel('Уровень частоты сердечных сокращений')
plt.ylabel('Количество')
plt.legend(title='Пол', labels=['Женщина', 'Мужчина'])

# Добавляем подписи под графиком
plt.text(0.0, 0, f'< {low_threshold}', ha='center', va='top', fontsize=10, color='green')
plt.text(1.0, 0, f'[{low_threshold} - {high_threshold}]', ha='center', va='top', fontsize=10, color='green')
plt.text(2.0, 0, f'> {high_threshold}', ha='center', va='top', fontsize=10, color='green')

plt.show()
```



У мужчин частоты сердечных сокращений больше, чем у женщин.

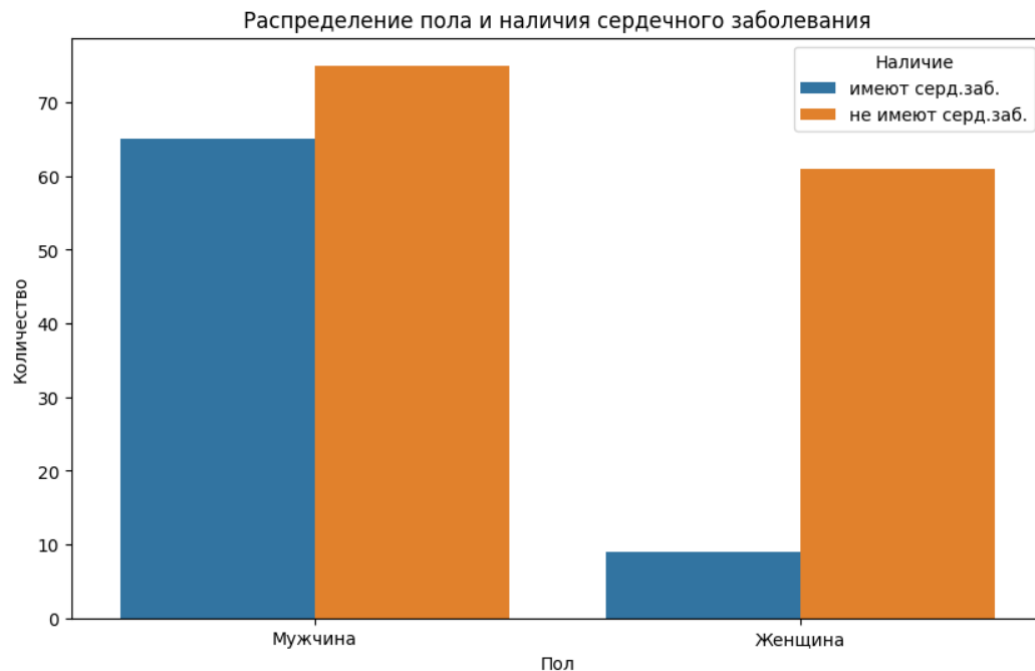
3.9. Распределение пола и наличия сердечных заболеваний

```
# Создаем график
plt.figure(figsize=(10, 6))
sb.countplot(x=df_data['sex'].replace({1: 'Мужчина', 0: 'Женщина'}), hue='target', data=df_data)

plt.title('Распределение пола и наличия сердечного заболевания')
plt.xlabel('Пол')
plt.ylabel('Количество')

# Добавление подписей
plt.legend(title='Наличие', labels=['имеют серд.заб.', 'не имеют серд.заб.'])

plt.show()
```



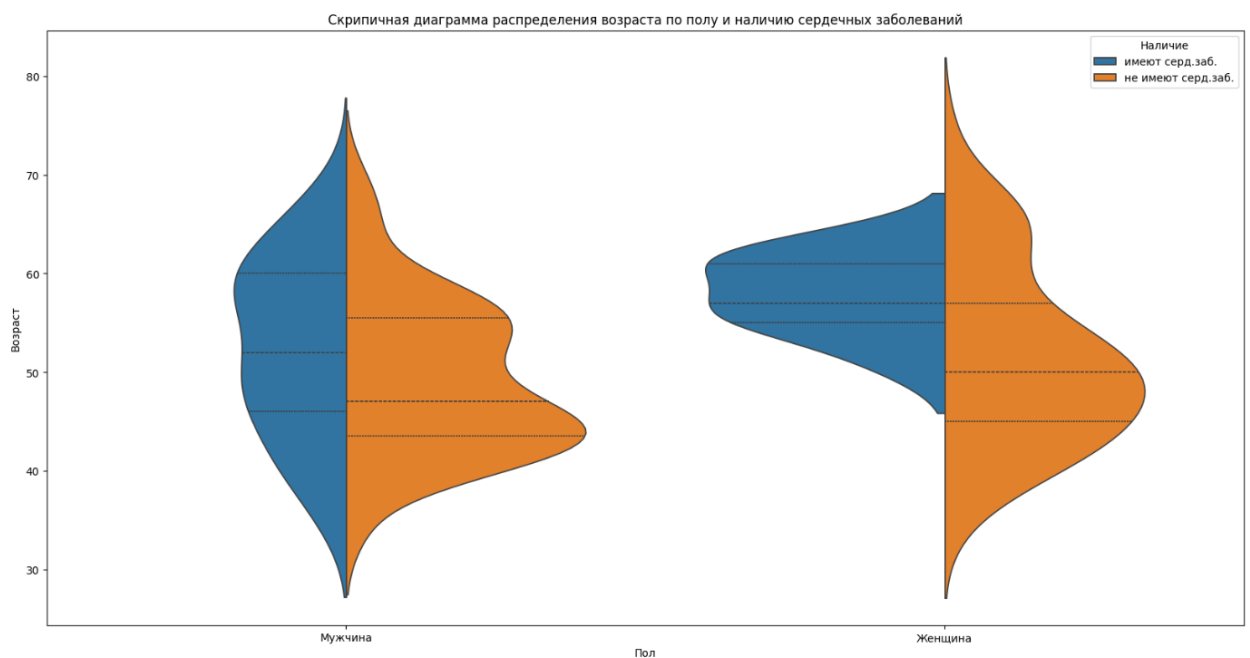
По графику видно, что большое количество у женщин и у мужчин не имеют сердечного заболевания, но у мужчин имеющиеся сердечные заболевания больше, чем у женщин.

Рассмотрим их на скрипичной диаграмме.

```
# Создаем график
plt.figure(figsize=(20, 10))
sb.violinplot(
    x=df_data['sex'].replace({1: 'Мужчина', 0: 'Женщина'}),
    y=df_data['age'],
    hue=df_data['target'].replace({0: 'имеют серд.заб.', 1: 'не имеют серд.заб.'}),
    split=True,
    inner='quartile'
)

plt.title('Скрипичная диаграмма распределения возраста по полу и наличию сердечных заболеваний')
plt.xlabel('Пол')
plt.ylabel('Возраст')

# Добавление подписей
plt.legend(title='Наличие')
plt.show()
```



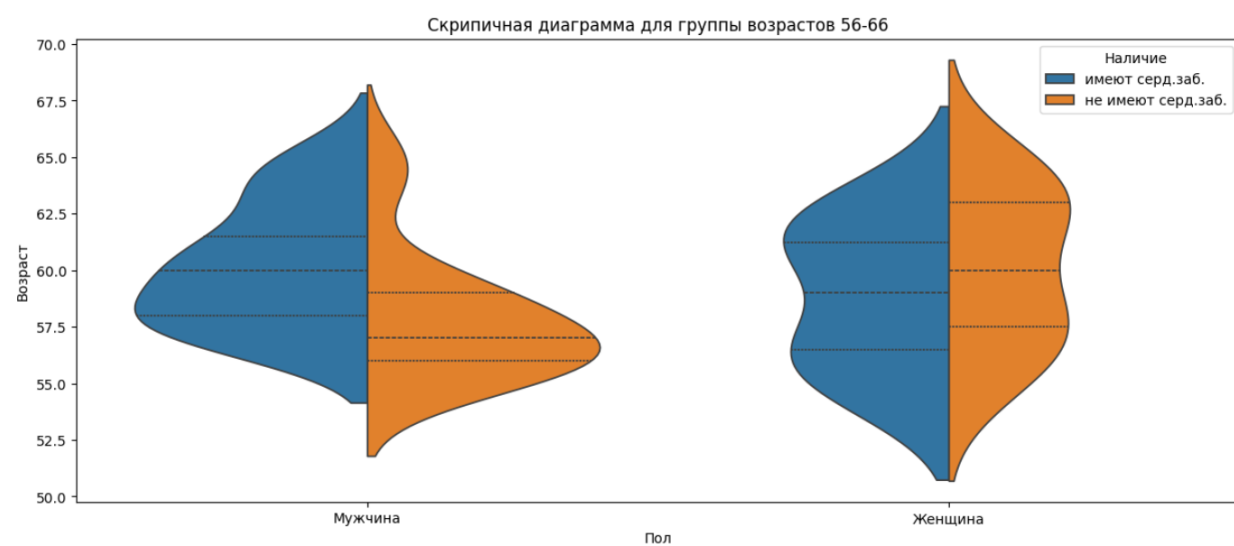
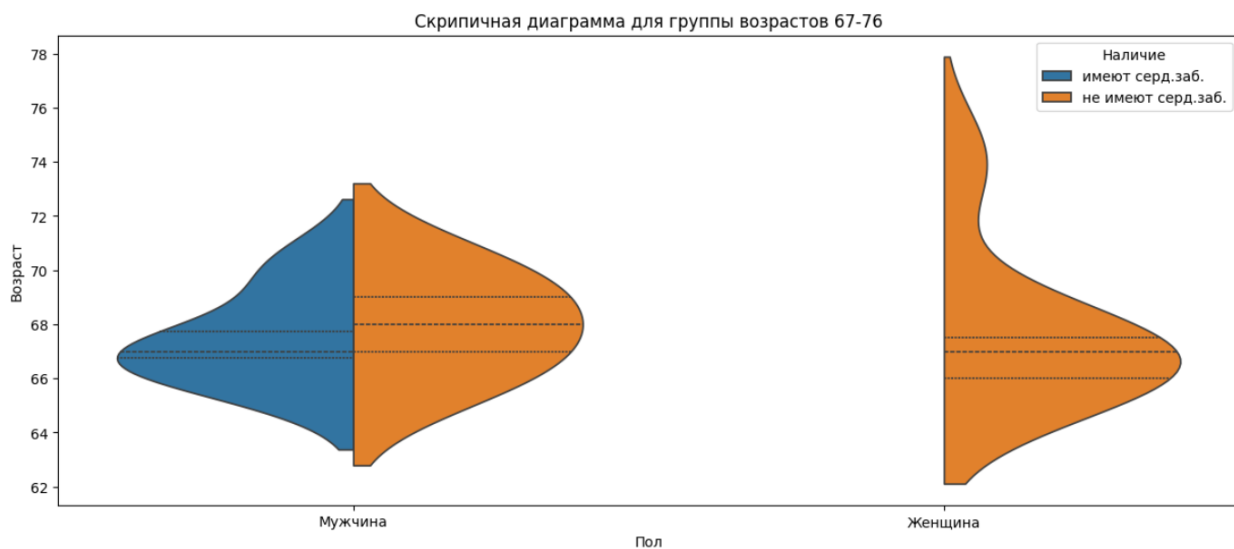
Женщины реже болеют, чем мужчины.

Рассмотрим по группам возрастов.

```
for age_group in df_data['age_group'].unique().sort_values(ascending=False):
    plt.figure(figsize=(15, 6))

    sb.violinplot(
        x=df_data['sex'].replace({1: 'Мужчина', 0: 'Женщина'}),
        y='age',
        hue=df_data['target'].replace({0: 'имеют серд.заб.', 1: 'не имеют серд.заб.'}),
        split=True,
        inner='quartile',
        data=df_data[df_data['age_group'] == age_group]
    )

    plt.title(f'Скрипичная диаграмма для группы возрастов {age_group}')
    plt.xlabel('Пол')
    plt.ylabel('Возраст')
    plt.legend(title='Наличие')
    plt.show()
```





По графику видно, что меньше данных доступно для женщин с сердечными заболеваниями в возрастных группах 34-44, 45-55 и 67-76 лет. Поэтому рассмотрим количество значений наличия сердечных заболеваний у мужчин и женщин.

```
# Замена числовых значений на текстовые метки
df_data['sex'] = df_data['sex'].map({1: 'Мужчина', 0: 'Женщина'})
df_data['target'] = df_data['target'].map({1: 'имеют серд.заб.', 0: 'Не имеют серд.заб.'})

# Подсчет количества значений target для каждого значения sex
counts = df_data.groupby('sex')['target'].value_counts()

# Вывод результата
print(counts)
```

sex	target	count
Женщина	имеют серд.заб.	61
	Не имеют серд.заб.	9
Мужчина	имеют серд.заб.	75
	Не имеют серд.заб.	65

Name: count, dtype: int64

Исходя из подсчета количества значений target для каждого значения sex, можно сделать следующие выводы: Среди женщин меньше случаев сердечных заболеваний по сравнению с мужчинами. В то же время, среди женщин больше случаев, когда сердечные заболевания отсутствуют, чем среди мужчин. Эти выводы могут быть полезны при анализе рисков сердечных заболеваний в зависимости от пола.